



Experiment 04

Custom Ordering & Hierarchical Aggregation in SQL

Name: Amrit

UID: 24BCC70009

Aim

To study and implement advanced SQL techniques such as conditional/custom ordering using CASE expressions inside ORDER BY, and self-joins for modelling hierarchical (manager–employee) relationships.

Question 1 (A)

You are given a table tbl_happiness that contains the happiness ranking of different countries.

- (i) Retrieve all the records from the table while ensuring that *India* and *Sri Lanka* appear at the top, followed by the rest of the countries in their original ranking order.
- (ii) Make all the rankings in descending order, except India and Sri Lanka.

Given Data

```
1 CREATE TABLE tbl_happiness
2 (
3     s_no      INT PRIMARY KEY,
4     rankings  INT,
5     country   VARCHAR (50)
6 );
7
8 INSERT INTO tbl_happiness VALUES
9 (1,1,'Finland'),
10 (2,2,'Denmark'),
11 (3,3,'Iceland'),
12 (4,4,'Israel'),
13 (5,5,'Netherlands'),
14 (6,6,'Sweden'),
15 (7,7,'Norway'),
16 (8,126,'India'),
17 (9,128,'Sri Lanka');
```

Answer – Part (i)

```
1 SELECT rankings, country
2
3 FROM tbl_happiness
4
5 ORDER BY
6 CASE WHEN country IN ('India','Sri Lanka') THEN 0 ELSE 1 END,
7 rankings;
```



Output – Part (i)

rankings	country
126	India
128	Sri Lanka
1	Finland
2	Denmark
3	Iceland
4	Israel
5	Netherlands
6	Sweden
7	Norway



**CHANDIGARH
UNIVERSITY**
Discover. Learn. Empower.

**NAAC
GRADE A+**
ACCREDITED UNIVERSITY



**CHANDIGARH
UNIVERSITY**
Discover. Learn. Empower.

Answer – Part (ii)

```
1 SELECT rankings , country
2
3 FROM tbl_happiness
4
5 ORDER BY
6 CASE WHEN country IN ('India','Sri Lanka') THEN 0 ELSE 1 END ,
7 CASE WHEN country IN ('India','Sri Lanka') THEN rankings END ASC ,
8 CASE WHEN country NOT IN ('India','Sri Lanka') THEN rankings END DESC
;
```

Output – Part (ii)

rankings	country
126	India
128	Sri Lanka
7	Norway
6	Sweden
5	Netherlands
4	Israel
3	Iceland
2	Denmark
1	Finland



Question 2 (C)

A company maintains employee information in a single table where each employee may report to another employee acting as their manager. Prepare a report that displays the details of every employee who supervises at least one other employee. The report should include the manager's employee ID, name, total number of direct reports, and the average age of those reporting employees. The average age should be rounded to the nearest whole number, and the final result should be sorted by employee ID.

Given Data

```

1 CREATE TABLE Employees
2 (
3     employee_id INT PRIMARY KEY,
4     name        VARCHAR (50) ,
5     reports_to  INT,
6     age         INT
7 );
8
9 INSERT INTO Employees VALUES
10 (9,'Hercy',NULL,43) ,
11 (6,'Alice',9,41),
12 (4,'Bob',9,36),
13 (2,'Winston',NULL,37);

```

Answer

```

1 SELECT
2 m.employee_id ,
3 m.name ,
4 COUNT(e.employee_id) AS reports_count ,
5 ROUND (AVG (e.age))  AS average_age
6
7 FROM Employees m
8
9 JOIN Employees e
10 ON e.reports_to = m.employee_id
11
12 GROUP BY m.employee_id , m.name
13
14 ORDER BY m.employee_id ;

```

Output

employee_id	name	reports_count	average_age
9	Hercy	2	39

Home Task

For additional practice on the concepts covered in this experiment, the following problems are recommended:



1. <https://datalemur.com/questions/product-line-profit>
2. <https://datalemur.com/questions/international-call-percentage>

Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

Understood how to implement custom/conditional sorting using CASE expressions inside ORDER BY.

Learned to prioritise specific rows while preserving the natural order of the remaining rows.

Implemented multi-level ORDER BY logic combining ascending and descending directions conditionally.

Applied self-joins to model manager–employee (hierarchical) relationships within a single table.

Used GROUP BY with COUNT() and ROUND(AVG()) to build a manager-wise summary report.

Strengthened practical understanding of PostgreSQL query design for real-world HR and ranking datasets.